

## One website. Eight Kubernetes skills.

A simple, guided Minikube lab for Ali Mansur. Allow about 90-120 minutes. Try each task using its hint first; use the worked example when you need help.

### What you will build

Run Nginx, change its home page through a ConfigMap, inject a dummy Secret, store a file on a PV/PVC, then reach the website through ClusterIP, NodePort and Ingress.

| Object              | Plain-language purpose                                           |
|---------------------|------------------------------------------------------------------|
| Deployment          | Keeps the requested number of application Pods running.          |
| Service / ClusterIP | Gives matching Pods a stable internal address.                   |
| ConfigMap           | Stores non-secret configuration or files.                        |
| Secret              | Stores sensitive data separately from application configuration. |
| PV                  | Represents storage available to the cluster.                     |
| PVC                 | Requests storage for a Pod to use.                               |
| NodePort Service    | Exposes a Service on a port on the node.                         |
| Ingress             | Defines HTTP host/path routing handled by a controller.          |

### Lab assumptions

Use Bash in Linux or WSL, with Docker running, kubectl and Minikube installed in the same environment. Minikube v1.39.0 or newer is needed for the Traefik addon. Use one node. Commands below use a separate profile named simple-k8s.

Create each named YAML file in a text editor, preserve spaces, then run the apply command. Do not paste YAML directly into the shell. These notes were checked against official documentation on 30 September 2026; the cluster was not executed here.

## Prepare the lab

**Your task:** Start a separate single-node cluster and create a namespace.

**Hint:** A profile separates this cluster from your other Minikube labs; a namespace groups its application objects.

```
minikube version
minikube start -p simple-k8s --driver=docker --nodes=1
kubectl config use-context simple-k8s
kubectl create namespace k8s-simple
kubectl config set-context --current --namespace=k8s-simple
kubectl get nodes
mkdir -p k8s-simple-lab
cd k8s-simple-lab
```

Run every later kubectl command in this context. The selected default namespace is k8s-simple, so you do not need to type -n k8s-simple each time. PVs are cluster-scoped; PVCs and the other application objects live in this namespace.

**Success check:** One node shows Ready. If Docker is unavailable, start Docker and retry. If the Traefik addon is unknown later, update Minikube to v1.39.0 or newer using its official installation instructions.

### Your working files

```
deployment.yaml
configmap.yaml
storage.yaml
service.yaml
nodeport.yaml
ingress.yaml
```

You will update deployment.yaml in later tasks. Keep the same file and object names throughout.

## Create a Deployment

**Your task:** Run one Nginx Pod using a Deployment called web.

**Hint:** The selector and Pod label must match: app: web. Nginx listens on port 80.

Save this as **deployment.yaml**.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
```

```
kubectl apply -f deployment.yaml
kubectl rollout status deployment/web --timeout=180s
kubectl get pods -l app=web
```

**Success check:** The Pod shows 1/1 Ready and Running. The stable image tag is convenient for this exercise; real releases should use a tested version or digest.

## Understand it

**replicas: 1** requests one Pod. **template** describes that Pod. **containerPort** documents the container port; it does not expose the application outside the cluster.

## Add a ConfigMap and a Secret

**Your task:** Create a custom home page and a dummy application token.

**Hint:** Use a ConfigMap for the HTML file. Use a Secret for the token. Creating either object alone does not attach it to a Pod.

Save as **configmap.yaml**.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-page
data:
  index.html: |
    <h1>Hello from my Kubernetes lab!</h1>
    <p>Deployment, ConfigMap and Service are working.</p>

kubectl apply -f configmap.yaml
kubectl create secret generic web-secret --from-literal=APP_TOKEN=practice-only-123
kubectl get configmap web-page
kubectl describe secret web-secret
```

**Success check:** ConfigMap web-page exists. The Secret lists APP\_TOKEN without showing its value. If the Secret already exists, keep it and continue.

### What happens next?

Task 4 mounts index.html into Nginx and injects APP\_TOKEN as an environment variable. The dummy token only demonstrates Secret injection; it does not add a login or password protection to the website.

Secret values are base64-encoded in API representations, which is not encryption. Use only the dummy value here; keep real credentials out of shared files and shell history. Access control and encryption at rest must be configured for real workloads.

## Create the PV and PVC

**Your task:** Create 1 GiB of lab storage and bind a claim to it.

**Hint:** PV = available storage; PVC = the application's request. Use the same capacity, access mode and storage class.

Save both objects in **storage.yaml**, separated by three hyphens.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: simple-web-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""
  hostPath:
    path: /data/simple-web
    type: DirectoryOrCreate
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ""
  volumeName: simple-web-pv
  resources:
    requests:
      storage: 1Gi

kubectl apply -f storage.yaml
kubectl get pv simple-web-pv
kubectl get pvc web-pvc
```

**Success check:** Both objects show Bound. An empty storageClassName disables the default dynamic storage class; volumeName selects this exact PV.

hostPath stores data on the Minikube node, not directly in your WSL working folder. Use it only for this single-node lab. The declared 1Gi does not enforce a filesystem quota. ReadWriteOnce means read/write from one node, not exactly one Pod.

## Attach configuration and storage

**Your task:** Update web to use the ConfigMap, Secret and PVC.

**Hint:** Mount the home page at the Nginx HTML path, but mount the PVC separately at /data so it does not hide the home page.

Replace **deployment.yaml** with this complete version.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
          env:
            - name: APP_TOKEN
              valueFrom:
                secretKeyRef:
                  name: web-secret
                  key: APP_TOKEN
          volumeMounts:
            - name: page
              mountPath: /usr/share/nginx/html
              readOnly: true
            - name: data
              mountPath: /data
      volumes:
        - name: page
          configMap:
            name: web-page
        - name: data
          persistentVolumeClaim:
            claimName: web-pvc
```

```
kubectl apply -f deployment.yaml
```

```
kubectl rollout status deployment/web --timeout=180s
```

```
kubectl exec deployment/web -- sh -c 'test -n "$APP_TOKEN" && echo "Secret is present"'
```

```
kubectl exec deployment/web -- cat /usr/share/nginx/html/index.html
```

**Success check:** The token-presence check succeeds and the HTML contains your custom greeting.

## Expose a ClusterIP Service

**Your task:** Create a stable internal Service named web.

**Hint:** Its selector must match app: web; targetPort must match the port Nginx listens on.

Save as **service.yaml**.

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - name: http
      port: 80
      targetPort: 80
```

```
kubectl apply -f service.yaml
kubectl get service web
kubectl get endpointslices -l kubernetes.io/service-name=web
kubectl run client --image=busybox:1.37 --restart=Never --rm -i -- wget -qO- http://web
```

**Success check:** The client prints your HTML. The Service has a ClusterIP and its EndpointSlice includes a Pod IP.

**port: 80** is the Service port. **targetPort: 80** is the application port in the Pod. ClusterIP is intended for internal cluster access; you normally cannot browse it directly from your laptop.

### Optional local browser check

```
kubectl port-forward service/web 8080:80
```

Leave that terminal open and visit **http://localhost:8080**. Press Ctrl+C to stop. Port-forward creates a temporary local tunnel; it does not change the Service to NodePort.

## Add a NodePort Service

**Your task:** Reach the same Pods through a node port.

**Hint:** Create a second Service with the same selector, but set type: NodePort and nodePort: 30080.

Save as **nodeport.yaml**. Keep the ClusterIP Service for the Ingress task.

```
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - name: http
      port: 80
      targetPort: 80
      nodePort: 30080
```

```
kubectl apply -f nodeport.yaml
kubectl get service web-nodeport
minikube -p simple-k8s service web-nodeport -n k8s-simple --url
```

**Success check:** Open the returned URL and see your greeting. Keep the command running if it creates a tunnel. A tunnel URL may use localhost and a different port; use the exact returned URL.

### If the node IP is directly reachable

```
minikube -p simple-k8s ip
curl http://$(minikube -p simple-k8s ip):30080
```

Windows/WSL with the Docker driver may not be able to route directly to the node IP. Use the Minikube service URL above. If a Windows browser cannot reach a WSL-local URL, run curl against that URL in another WSL terminal.

The node receives traffic on **30080**; the Service port is **80**; Nginx receives traffic on **80**. The default NodePort range is 30000-32767.

## Route through Ingress

**Your task:** Route requests for web.test to the web Service.

**Hint:** An Ingress is a routing rule. Install a controller to implement the rule. The backend uses Service port 80, not NodePort 30080.

```
minikube -p simple-k8s addons enable traefik
kubectl -n kube-system rollout status deployment/traefik --timeout=180s
kubectl get ingressclass
```

Confirm class **traefik** exists. Save as **ingress.yaml**.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
spec:
  ingressClassName: traefik
  rules:
    - host: web.test
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

```
kubectl apply -f ingress.yaml
kubectl get ingress web
kubectl -n kube-system port-forward service/traefik 8081:80
```

Keep this terminal open. In a second terminal, test the host rule:

```
curl -H "Host: web.test" http://127.0.0.1:8081/
```

**Success check:** You receive the greeting. This tunnel reaches the controller, so the Ingress rule is actually tested. Forwarding service/web would bypass Ingress.

Where the node IP is reachable, also try:

```
curl -H "Host: web.test" http://$(minikube -p simple-k8s ip)/
```

For browser access through the tunnel, map **127.0.0.1 web.test** in the hosts file of the OS running your browser, then open **http://web.test:8081**. The curl test needs no hosts-file change. This lab uses HTTP; TLS is outside its scope.

## Prove that storage survives

**Your task:** Write a file, replace the Pod, and read the file from its replacement.

**Hint:** Write under /data, which is backed by the PVC. Do not delete the cluster or PVC.

```
kubectl exec deployment/web -- sh -c 'echo "My data survived" && /data/note.txt'
kubectl get pods -l app=web
kubectl delete pod -l app=web
kubectl rollout status deployment/web --timeout=180s
kubectl get pods -l app=web
kubectl exec deployment/web -- cat /data/note.txt
```

**Success check:** The Pod name changes, but the file still says My data survived. The Deployment recreated the Pod; the PVC reattached the same storage.

### Small extra challenge: change the page

Change the greeting in configmap.yaml, then run:

```
kubectl apply -f configmap.yaml
kubectl rollout restart deployment/web
kubectl rollout status deployment/web --timeout=180s
```

Recheck NodePort or Ingress and refresh the browser. A full ConfigMap directory mount normally updates after a delay; this restart gives a clear, repeatable checkpoint. Secret environment variables also require Pod replacement to pick up changed values.

### Explain your project in an interview

"I deployed Nginx using a Deployment. A ConfigMap supplied the page, and a Secret supplied a dummy environment variable. A PVC bound to a PV preserved a file when the Pod was replaced. ClusterIP provided internal access, NodePort exposed a node port, and a Traefik controller implemented my Ingress host rule."

### Remember the traffic paths

**NodePort:** client reaches the node port, which forwards to a matching application Pod.

**Ingress:** client reaches the controller; the host/path rule identifies the backend Service and its application endpoints. The separate web-nodeport Service is not required for this Ingress route.

## Troubleshooting and cleanup

### Investigate in this order

```
kubectl get pods,pvc,svc,ingress
kubectl describe deployment web
kubectl describe pod <pod-name>
kubectl logs deployment/web
kubectl get events --sort-by=.metadata.creationTimestamp
```

Replace <pod-name> with the actual name from `kubectl get pods`. Read the Events section of describe before changing anything.

**Pod Pending:** Run `kubectl describe pvc web-pvc`. Confirm the PV name, capacity, access mode and empty storageClassName match.

**CreateContainerConfigError:** Confirm web-secret exists in k8s-simple and contains the key APP\_TOKEN.

**FailedMount:** Confirm web-page and web-pvc exist. Check Pod Events for the exact missing resource.

**ImagePullBackOff:** Check image spelling, internet access and registry availability in Pod Events.

**Service does not respond:** Check selector app: web, EndpointSlices, Pod readiness and targetPort: 80.

**Nginx returns 403:** Verify the mounted ConfigMap includes index.html. Do not mount the empty PVC over /usr/share/nginx/html.

**Ingress returns 404:** Send Host: web.test. Check ingressClassName: traefik and that the controller is running.

**Node IP cannot be reached:** Use the Minikube service URL for NodePort or the controller port-forward for Ingress.

### Cleanup - only after finishing

Stop active tunnels with Ctrl+C. The following removes this lab's application objects and PV definition. Retain leaves storage data on the node.

```
kubectl delete namespace k8s-simple
kubectl delete pv simple-web-pv
minikube stop -p simple-k8s
```

If you want to permanently remove the entire lab cluster and its local data, use **minikube delete -p simple-k8s**. Remove any web.test hosts-file entry you added.

## Official references and final checklist

Documentation checked on 30 September 2026. This is a guided local exercise, not a production deployment template.

### Deployment

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

### ConfigMap

<https://kubernetes.io/docs/concepts/configuration/configmap/>

### Secret

<https://kubernetes.io/docs/concepts/configuration/secret/>

### PV/PVC tutorial

<https://kubernetes.io/docs/tutorials/configuration/configure-persistent-volume-storage/>

### Services and NodePort

<https://kubernetes.io/docs/concepts/services-networking/service/>

### Ingress

<https://kubernetes.io/docs/concepts/services-networking/ingress/>

### Minikube Traefik addon

<https://minikube.sigs.k8s.io/docs/handbook/addons/traefik/>

### Minikube installation

<https://minikube.sigs.k8s.io/docs/start/>

### Ingress NGINX retirement

<https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/>

Controller choice: Ingress NGINX retired in March 2026. The current Minikube documentation recommends the Traefik addon. The Nginx application in this lab is a separate web server and is unaffected by that controller retirement.

## Can you demonstrate every outcome?

- [ ] A Deployment maintains a running Nginx Pod.
- [ ] The ConfigMap provides the custom home page.
- [ ] The dummy Secret is injected without printing its value.
- [ ] PV and PVC are Bound, and data survives Pod replacement.
- [ ] ClusterIP works from the temporary client Pod.
- [ ] NodePort works using the Minikube service URL.
- [ ] Ingress routes Host: web.test through the controller.